

Strings

In computer science, sequences of characters are referred to as *strings*. Strings can be any length and can include any character such as letters, numbers, symbols, and whitespace (spaces, tabs, new lines).

Escaping Characters

Backslashes (\) are used to escape characters in a Python string.
For instance, to print a string with quotation marks, the given code snippet can be used.

```
txt = "She said \"Never let go\"."
print(txt) # She said "Never let go".
```

The in Syntax

The `in` syntax is used to determine if a letter or a substring exists in a string. It returns `True` if a match is found, otherwise `False` is returned.

```
game = "Popular Nintendo Game: Mario
Kart"

print("l" in game) # Prints: True
print("x" in game) # Prints: False
```

Indexing and Slicing Strings

Python strings can be indexed using the same notation as lists, since strings are lists of characters. A single character can be accessed with bracket notation (`[index]`), or a substring can be accessed using slicing (`[start:end]`).
Indexing with negative numbers counts from the end of the string.

```
str = 'yellow'

str[1]      # => 'e'
str[-1]     # => 'w'
str[4:6]    # => 'ow'
str[:4]     # => 'yell'
str[-3:]    # => 'low'
```

Iterate String

To iterate through a string in Python, “for...in” notation is used.

```
str = "hello"
for c in str:
    print(c)
```

```
# h
# e
# l
# l
# o
```

Built-in Function len()

In Python, the built-in `len()` function can be used to determine the length of an object. It can be used to compute the length of strings, lists, sets, and other countable objects.

```
length = len("Hello")
print(length)
# Output: 5
```

```
colors = ['red', 'yellow', 'green']
print(len(colors))
# Output: 3
```

String Concatenation

To combine the content of two strings into a single string, Python provides the `+` operator. This process of joining strings is called concatenation.

```
x = 'One fish, '
y = 'two fish.'

z = x + y

print(z)
# Output: One fish, two fish.
```

Immutable strings

Strings are immutable in Python. This means that once a string has been defined, it can't be changed.

There are no mutating methods for strings. This is unlike data types like lists, which can be modified once they are created.

IndexError

When indexing into a string in Python, if you try to access an index that doesn't exist, an `IndexError` is generated. For example, the following code would create an `IndexError` :

```
fruit = "Berry"  
indx = fruit[6]
```

